LESSON 9

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Reaction Timer

Drive a 4-digit display with a clever trick and test how fast you are.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How **multiplexing** fools your eyes into seeing 4 digits at once
- How to use **millis()** as a stopwatch instead of `delay()`
- How to split a number into digits with **/** and **%**
- How to build a game with **while** loops

SESSION PLAN

0:00	How can 4 digits share the same wires?
0:10	Wire up the 4-digit display
0:40	Sketch 1: **L09_Multiplex**
0:55	Short break
1:00	Sketch 2: **L09_Counter** (millis and digits)
1:20	Sketch 3: **L09_Reaction** (the game!)
1:40	Challenges: beat your friends, catch cheaters
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable, on your 2 breadboards
- ☐ 4-digit 7-segment display (12 pins)
- ☐ 8 resistors, 220 Ω (red-red-brown)
- ☐ 1 push button
- ☐ About 16 jumper wires

1. Four digits, one trick

The 4-digit display has **4 × 8 = 32** little LEDs, but only **12 pins**. How?

All four digits **share** the same 8 segment wires (a–g and dp). Each digit also has its own **digit pin**, which is its shared **–** leg. A digit only lights up when its digit pin is **LOW** (connected to 0 V).

So the ESP32 does this, over and over:

1. Switch on the bars for the **1st** digit, light only digit 1, then wait a tiny moment
2. Switch on the bars for the **2nd** digit, light only digit 2, then wait a tiny moment
3. … digit 3 … digit 4 … then back to the start

> **Big idea: multiplexing**
>
> Only **one** digit is ever lit at a time! But the ESP32 switches between them about **100 times a second**. That's too fast for your eyes, so you see all four at once. Films and TV screens fool your eyes in a similar way.

Hold the display with the **decimal points at the bottom**. Pin 1 is at the bottom left:

```
  pin: 12  11  10   9   8   7
       D1   a   f  D2  D3   b       (top row)
      +-------------------------+
      |   8.   8.   8.   8.     |
      +-------------------------+
       e   d  dp   c   g  D4       (bottom row)
  pin: 1   2   3   4   5   6
```

2. Wire it up

Push the display across the middle gap of a breadboard. You can reuse the ESP32 wires from Lesson 7: the **segment pins on the ESP32 are the same**. Only the display's pin positions are different.

Segments: each one goes through a 220 Ω resistor.

✓	ESP32 pin	goes to	Display pin	Segment
☐	**23**	220 Ω resistor →	pin 11	a
☐	**22**	220 Ω resistor →	pin 7	b
☐	**21**	220 Ω resistor →	pin 4	c
☐	**19**	220 Ω resistor →	pin 2	d
☐	**18**	220 Ω resistor →	pin 1	e
☐	**17**	220 Ω resistor →	pin 10	f
☐	**16**	220 Ω resistor →	pin 5	g
☐	**4**	220 Ω resistor →	pin 3	dp

Digits: these connect straight to the ESP32 with a jumper wire, *without* resistors.

✓	ESP32 pin	Display pin	Digit
☐	**25**	pin 12	D1 (left)
☐	**26**	pin 9	D2
☐	**27**	pin 8	D3
☐	**32**	pin 6	D4 (right)

Button: one leg to ESP32 pin **13**, the diagonally opposite leg to the **–** rail. Connect ESP32 **GND** to the **–** rail.

3. Sketch 1: See the trick

Type this in and save it as **L09_Multiplex**. The digit table is the same as in Lesson 7. If you still have L07_Count, you can copy and paste it!

Type this in → save it as **L09_Multiplex** 49 lines

```
// Lesson 9: Four digits, one at a time
// Shows 1234 by lighting each digit in turn, very fast.

const int segPins[7] = {23, 22, 21, 19, 18, 17, 16};
const int digitPins[4] = {25, 26, 27, 32};

const int digits[10][7] = {
// a  b  c  d  e  f  g
  {1, 1, 1, 1, 1, 1, 0},  // 0
  {0, 1, 1, 0, 0, 0, 0},  // 1
  {1, 1, 0, 1, 1, 0, 1},  // 2
  {1, 1, 1, 1, 0, 0, 1},  // 3
  {0, 1, 1, 0, 0, 1, 1},  // 4
  {1, 0, 1, 1, 0, 1, 1},  // 5
  {1, 0, 1, 1, 1, 1, 1},  // 6
  {1, 1, 1, 0, 0, 0, 0},  // 7
  {1, 1, 1, 1, 1, 1, 1},  // 8
  {1, 1, 1, 1, 0, 1, 1}   // 9
};

void showDigit(int place, int n) {
  for (int d = 0; d < 4; d++) {
    digitalWrite(digitPins[d], HIGH);  // all digits off
  }
  for (int s = 0; s < 7; s++) {
    digitalWrite(segPins[s], digits[n][s]);
  }
  digitalWrite(digitPins[place], LOW);  // this digit on
}

void setup() {
  for (int s = 0; s < 7; s++) {
    pinMode(segPins[s], OUTPUT);
  }
  for (int d = 0; d < 4; d++) {
    pinMode(digitPins[d], OUTPUT);
  }
}

void loop() {
  showDigit(0, 1);
```

```
  delay(5);
  showDigit(1, 2);
  delay(5);
  showDigit(2, 3);
  delay(5);
  showDigit(3, 4);
  delay(5);
}
```

You should see **1234**.

Code	What it means
`const int digitPins[4]`	The 4 pins that choose **which digit** is lit
`void showDigit(int place, int n)`	Show number **n** at position **place** (0 = left, 3 = right)
`digitalWrite(digitPins[d], HIGH);`	**HIGH** switches a digit **off**, because its – leg isn't at 0 V any more
`digitalWrite(digitPins[place], LOW);`	**LOW** switches just this one digit **on**

Try it: catch the trick!

1. Change all four `delay(5)` to `delay(200)`. Now you can see what is really happening!
2. Try `delay(20)`. Does it flicker? Find the biggest delay that still looks steady to you:

   _____
3. Make it show your birth year, or **4321**.

Some segments look dim or wrong?

Check the segment table. Remember, the segment pins on this display are in **different places** from the Lesson 7 display. If one whole digit is dark, check its digit wire.

4. Sketch 2: A seconds counter

To show a number like **2735**, we must split it into four digits. Two maths symbols help:

Symbol	Meaning	Example
`/`	Divide, and **throw away** the remainder	`2735 / 100` is **27**
`%`	Just the **remainder** after dividing	`27 % 10` is **7**

So `2735 / 1000 % 10` is 2, `2735 / 100 % 10` is 7, `2735 / 10 % 10` is 3, and `2735 / 1 % 10` is 5.

File → Save As… your L09_Multiplex sketch with the new name **L09_Counter**. Then make two changes. Everything else stays the same!

Change 1: add this new function just **below** `showDigit`:

```
void showNumber(int number) {
  number = constrain(number, 0, 9999);
  int divisor = 1000;
  for (int place = 0; place < 4; place++) {
    showDigit(place, number / divisor % 10);
    delay(2);
    divisor = divisor / 10;
  }
}
```

Change 2: replace the whole `loop()` with this much shorter one:

```
void loop() {
  int seconds = millis() / 1000;
  showNumber(seconds);
}
```

Code	What it means
`millis()`	How many **milliseconds** since the ESP32 started. It's like a stopwatch that never stops
`constrain(number, 0, 9999)`	Keep the number between 0 and 9999, because 4 digits can't show more
`divisor = divisor / 10;`	1000 → 100 → 10 → 1: one step for each digit

> **Big idea: why not use delay()?**
>
> `delay(1000)` would freeze the ESP32 for a whole second. Nothing could be shown on the display, so it would go **dark**! With `millis()` the ESP32 just *checks the time* and keeps on refreshing the display.

> **Try it**
>
> Change `millis() / 1000` to `millis() / 100`. What is it counting now? Then press the **EN** button. What happens?

5. Sketch 3: Reaction timer game

How fast are you? The display shows **0** for a random time. Then it starts counting up in milliseconds. Press the button as fast as you can to stop it!

Save a copy as **L09_Reaction** and add the new parts:

Type this in → save it as **L09_Reaction** 69 lines

```
// Lesson 9: Reaction timer game
// When the numbers start counting, press the button as fast as you can!

const int BUTTON_PIN = 13;
const int segPins[7] = {23, 22, 21, 19, 18, 17, 16};
const int digitPins[4] = {25, 26, 27, 32};

const int digits[10][7] = {
// a  b  c  d  e  f  g
  {1, 1, 1, 1, 1, 1, 0},  // 0
  {0, 1, 1, 0, 0, 0, 0},  // 1
  {1, 1, 0, 1, 1, 0, 1},  // 2
  {1, 1, 1, 1, 0, 0, 1},  // 3
  {0, 1, 1, 0, 0, 1, 1},  // 4
  {1, 0, 1, 1, 0, 1, 1},  // 5
  {1, 0, 1, 1, 1, 1, 1},  // 6
  {1, 1, 1, 0, 0, 0, 0},  // 7
  {1, 1, 1, 1, 1, 1, 1},  // 8
  {1, 1, 1, 1, 0, 1, 1}   // 9
};

void showDigit(int place, int n) {
  for (int d = 0; d < 4; d++) {
    digitalWrite(digitPins[d], HIGH);  // all digits off
  }
  for (int s = 0; s < 7; s++) {
    digitalWrite(segPins[s], digits[n][s]);
  }
  digitalWrite(digitPins[place], LOW);  // this digit on
}

void showNumber(int number) {
  number = constrain(number, 0, 9999);
  int divisor = 1000;
  for (int place = 0; place < 4; place++) {
    showDigit(place, number / divisor % 10);
    delay(2);
    divisor = divisor / 10;
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  for (int s = 0; s < 7; s++) {
    pinMode(segPins[s], OUTPUT);
  }
```

```
  for (int d = 0; d < 4; d++) {
    pinMode(digitPins[d], OUTPUT);
  }
}

void loop() {
  unsigned long goTime = millis() + random(2000, 6000);
  while (millis() < goTime) {
    showNumber(0);  // get ready...
  }
  while (digitalRead(BUTTON_PIN) == HIGH) {
    showNumber(millis() - goTime);  // GO! count until pressed
  }
  int score = millis() - goTime;
  Serial.print("Your time: ");
  Serial.print(score);
  Serial.println(" ms");
  unsigned long showUntil = millis() + 3000;
  while (millis() < showUntil) {
    showNumber(score);
  }
}
```

Code	What it means
`unsigned long goTime`	A **long** number for big times. **unsigned** means it is never negative
`millis() + random(2000, 6000)`	"Now, plus 2 to 6 seconds", so nobody knows when it will start
`while (millis() < goTime)`	Keep showing 0 until it's time to go
`while (digitalRead(BUTTON_PIN) == HIGH)`	Keep counting **while** the button is not pressed

6. Challenges

Challenge 1: Catch the cheater

What happens if you hold the button down **before** the counting starts? Fix the game so it shows **9999** for cheaters. *Hint: check the button inside the "get ready" loop.*

Challenge 2: Best score

Make a variable `best` that remembers the fastest time. After each go, print `"New record!"` in the Serial Monitor if you beat it. Then have a competition with your friends: 3 tries each!

Challenge 3: Stopwatch

Make a stopwatch: press once to start, press again to stop, and press a third time to reset. Show tenths of a second, and switch on the **decimal point** to show **12.3**.

7. Quiz

Check what you learned

1. In multiplexing, how many digits are lit at exactly the same moment?

   ---

2. What are `4567 / 100` and `4567 % 10`?

   ---

3. Why does `delay(1000)` make the 4-digit display go dark?

Tidy up: unplug the USB cable. Take everything off the breadboard except the ESP32. Next lesson uses a screen!